EPBFL: Efficient Privacy-Preserving Byzantine-Robust Federated Learning with Top-k Sparsification

Taihao Zhang, Wenting Shen, Hanlin Zhang, Ye Su, and Jing Qin

***Abstract*—Federated learning (FL) is vulnerable to Byzantine attacks and model privacy leakage. Existing Byzantine-robust methods may suffer from unreliable detection under heterogeneous data distributions or rely on trusted reference data that are not always available. Homomorphic encryption can protect model updates but incurs substantial overhead for high-dimensional models. Although Top-k sparsification reduces this overhead, independently sparsified model updates retain different parameter indices and must be aligned for correct aggregation and pairwise dissimilarity computation. The challenge is to achieve such alignment without revealing individual clients' retained parameter index sets while supporting aggregation and pairwise dissimilarity computation over encrypted model updates. To address these challenges, we propose EPBFL, an efficient privacy-preserving Byzantine-robust FL framework with Top-k sparsification. EPBFL employs masked block-wise P-ary encoding to recover aggregate parameter-index frequencies without revealing individual clients' retained parameter index sets. Based on the recovered frequencies, a global parameter index sequence is constructed to form compact model updates, reducing the overhead of subsequent encrypted computation. The compact model updates are encrypted using segmented CKKS, and a Bray–Curtis (BC)-inspired dissimilarity is designed to enable efficient pairwise dissimilarity computation over the encrypted compact model updates. Based on the resulting pairwise dissimilarity matrix, density-based clustering and cluster-wise weighted aggregation are performed to mitigate Byzantine attacks under heterogeneous data distributions without relying on trusted reference data. Experiments on MNIST, Fashion-MNIST, and CIFAR-10 under IID and non-IID settings demonstrate that EPBFL achieves the best or competitive test accuracy in most settings across different Byzantine attacks. EPBFL also achieves substantially lower communication and computational overhead than existing privacy-preserving Byzantine-robust FL schemes.**

***Index Terms*—Article submission, IEEE,**

I. Introduction

Federated Learning (FL) [1] is a distributed machine learning paradigm that enables multiple clients to collaboratively train a global model under the coordination of a central server. In each training round, clients compute local model updates using their private data and transmit the updates to the server for aggregation, thereby avoiding direct sharing of raw data. However, despite its decentralized architecture, FL is inherently vulnerable to Byzantine attacks [2], [3]. Specifically, compromised clients may submit malicious model updates to manipulate the aggregation process [4] and degrade model performance or even prevent convergence of the global model [5], [6].

To defend against Byzantine attacks, numerous Byzantine-robust FL schemes have been proposed [7], [8], [9]. Most existing methods detect malicious model updates by measuring pairwise relationships among client model updates using geometric metrics, such as Euclidean distance [10], [11] and cosine similarity [12], [13]. However, under heterogeneous data distributions, benign model updates may naturally differ significantly due to statistical heterogeneity. As a result, benign model updates may be misclassified as malicious. To improve detection reliability, some methods further leverage a small trusted dataset maintained by the server to evaluate client updates or construct trusted references for anomaly detection [14], [15]. Although such reference-based methods can alleviate the impact of data heterogeneity, obtaining a representative trusted dataset is often impractical due to privacy constraints and limited data availability. Therefore, achieving reliable Byzantine detection under heterogeneous data distributions without trusted reference data remains challenging.

Besides detection reliability, preserving model privacy further complicates Byzantine-robust FL. Existing privacy-preserving Byzantine-robust FL schemes mainly rely on differential privacy (DP) [16], [17], secure multi-party computation (SMPC) [18], [19], and homomorphic encryption (HE) [20]. DP [16] may obscure the distinctions between benign and malicious updates, while SMPC [18], [19] often incurs substantial communication overhead. Compared with DP and SMPC, HE enables computation directly over encrypted model updates without revealing their plaintext values, making it a promising solution for privacy-preserving Byzantine-robust FL. However, applying HE to Byzantine detection remains computationally expensive because modern deep neural networks usually contain millions of parameters. In particular, computing pairwise relationships among high-dimensional encrypted model updates requires substantial ciphertext computation and communication overhead.

To reduce such overhead, Top-k sparsification [21], [22] can be used to compress model updates before encryption. Specifically, each client independently retains only a fraction k of the d model parameters with the largest magnitudes, resulting in kd retained parameters and thereby reducing the overhead of subsequent encryption and pairwise dissimilarity computation [17]. However, different clients generally retain different parameter indices after Top-k sparsification. Thus, their sparse model updates cannot be directly aligned, making

This research is supported by National Natural Science Foundation of China (U24A20244), Shandong Provincial Natural Science Foundation, China (ZR2025MS1006) and Youth Innovation Technology Project of Higher School in Shandong Province (2023KJ365).

T. Zhang, W. Shen, and H. Zhang are with the College of Computer Science and Technology, Qingdao University, Qingdao 266071, China. E-mail: gojeonpa9938@163.com, shenwentingmath@163.com, hanlin10001@gmail.com. (*Corresponding author: W. Shen*)

Y. Su is with the School of Info rmation Science and Engineering, Shandong Normal University, Jinan 250014, China. E-mail: suye@sdnu.edu.cn.

J. Qin is with the School of Mathematics, Shandong University, Jinan, 250100, China. E-mail: qinjing@sdu.edu.cn.

it difficult to correctly aggregate model update values or compute pairwise dissimilarities between clients. Therefore, parameter index alignment is required before pairwise dissimilarity computation can be performed over encrypted sparse model updates.

A straightforward way to align sparse model updates is to transmit the retained parameter indices together with their corresponding values. For example, PrivCrFL [18] uploads sparse tuples containing parameter indices and parameter values, allowing the server to identify the corresponding parameters across clients. However, recent studies [17], [23] have shown that retained parameter indices may also reveal sensitive information about local training data. Therefore, directly transmitting the retained parameter indices introduces additional privacy leakage, whereas hiding them makes it difficult for the server to align sparse model updates. Without parameter index alignment, pairwise dissimilarity computation over encrypted sparse model updates cannot be correctly performed. Therefore, *a key challenge is how to align sparse model updates with different retained parameter indices without revealing individual clients' retained parameter index sets, while still supporting efficient pairwise dissimilarity computation over encrypted model updates for Byzantine detection.*

To address this challenge, we propose EPBFL, an efficient privacy-preserving Byzantine-robust FL framework that enables encrypted Byzantine detection over independently sparsified model updates. EPBFL first derives a global parameter index sequence without revealing individual clients' retained parameter index sets and uses it to construct aligned compact model updates. Based on these aligned compact model updates, EPBFL securely computes pairwise dissimilarities for Byzantine detection and cluster-wise weighted aggregation under heterogeneous data distributions.

The main contributions are summarized as follows:

- We propose a lightweight privacy-preserving parameter index alignment method for Top-k sparse model updates. Retained parameter indices are represented using masked block-wise P-ary encodings, enabling the aggregation server to recover aggregate parameter-index frequencies without revealing clients' retained parameter index sets. Based on the recovered frequencies, a global parameter index sequence is constructed to align sparse model updates across clients for encrypted Byzantine detection.
- Based on aligned compact model updates, We introduce a Bray-Curtis (BC)-inspired dissimilarity for efficient pairwise comparison under segmented CKKS encryption. By replacing the absolute-value operations in the conventional BC dissimilarity with squared terms, the proposed dissimilarity can be efficiently computed over encrypted compact model updates using homomorphic addition and multiplication.
- Based on the resulting pairwise dissimilarity matrix, we develop a density-based Byzantine detection and cluster-wise weighted aggregation mechanism. EPBFL forms multiple density-connected clusters and excludes isolated outliers without relying on trusted reference data. Within each retained cluster, clients are adaptively weighted according to their intra-cluster dissimilarity, reducing the influence of inconsistent model updates under heterogeneous data distributions.
- Extensive experiments across three benchmark datasets, four representative Byzantine attacks, and both IID and non-IID data distributions demonstrate the robustness and efficiency of EPBFL. The results show that EPBFL maintains high test accuracy under Byzantine attacks while significantly reducing communication and computational overhead.

The rest of this paper is organized as follows. Section II reviews the related work, followed by the preliminaries in Section III. The system and threat models are presented in Section IV, and the proposed EPBFL framework is described in Section V. Section VI provides the theoretical analysis of EPBFL, whereas its performance is evaluated in Section VII. Finally, Section VIII concludes the paper.

II. Related Work

Federated learning is vulnerable to Byzantine attacks, in which malicious clients intentionally manipulate the training process through data poisoning or model poisoning by corrupting local datasets or local model updates [24]. To improve robustness against such attacks, numerous Byzantine-robust federated learning schemes are available. Krum [24] selects the model updates with the smallest sum of Euclidean distances to its nearest neighbors. Median [25] performs coordinate-wise aggregation to suppress the influence of abnormal model updates. AFA [26] uses cosine similarity to evaluate the consistency between local model updates and a trusted reference. FLTrust [27] assumes that the server possesses a small trusted dataset to generate a reliable reference model update for identifying malicious clients. More recent methods, such as FLAD [8] and FoundationFL [28], reduce the reliance on trusted reference data by exploiting similarity relationships among client model updates. Although these methods achieve varying degrees of robustness against Byzantine attacks, most of them rely on geometric similarity metrics or trusted reference model updates for malicious client detection. Their detection effectiveness may degrade under heterogeneous data distributions, where benign model updates naturally exhibit significant statistical variations.

To protect the privacy of client model updates during Byzantine detection, recent studies employ differential privacy (DP) [16], [17], secure multi-party computation (SMPC) [18], [19], and homomorphic encryption (HE) [12], [20], often combining multiple techniques within a hybrid design. DP-based methods [16] introduce random perturbations that may distort the structural characteristics of model updates and reduce the reliability of similarity-based Byzantine detection. SMPC-based approaches [19] require frequent interactions among clients, resulting in considerable communication overhead. In contrast, HE enables direct computation over encrypted model updates without exposing raw information, making it particularly suitable for privacy-preserving Byzantine-robust federated learning. ShieldFL [12] adopts a dual-server architecture with two-trapdoor homomorphic encryption and computes cosine similarity to detect

suspicious model updates in the encrypted domain. RFed [11] leverages CKKS encryption together with a scaled dot-product attention mechanism to assign adaptive aggregation weights to client model updates. PEFL [29] combines linearly homomorphic encryption with median-based benchmarking and Pearson correlation for robust aggregation. Although these studies demonstrate the feasibility of Byzantine detection over encrypted model updates, they generally perform encrypted computation on dense model updates, leading to substantial computational and communication overhead for high-dimensional models.

To improve the efficiency of privacy-preserving Byzantine-robust FL, recent studies [30], [31] explore dimensionality reduction and model sparsification. Qin et al. [31] combine homomorphic encryption with Johnson-Lindenstrauss-based dimensionality reduction to reduce the cost of encrypted similarity computation. However, since dimensionality reduction is applied only after encrypted model updates are uploaded to the server, it cannot effectively reduce the communication overhead incurred by transmitting high-dimensional encrypted model updates. Among existing model sparsification techniques, Top-k sparsification attracts considerable attention because it reduces communication and computation overhead by retaining only a subset of parameters with the largest magnitudes [21], [22].

However, because different clients generally retain different parameter indices, sparse model updates cannot be directly aligned across clients. One line of research addresses this issue by transmitting the retained parameter indices together with their corresponding parameter values. For example, PrivCrFL [18] uploads sparse tuples consisting of parameter indices and parameter values, allowing the server to align sparse model updates across clients. However, exposing retained parameter indices may leak sensitive information about local training data [23]. Existing privacy-preserving sparsification schemes [17], [23] also consider protecting retained parameter indices, but mainly focus on secure sparse aggregation or communication efficiency. In contrast, Byzantine detection requires evaluating relationships among client model updates, which becomes more challenging when sparse model updates are encrypted and their parameter indices are protected. Therefore, supporting privacy-preserving parameter index alignment together with efficient pairwise dissimilarity computation over encrypted sparse model updates remains challenging.

III. Preliminaries

A. Federated Learning

Federated learning (FL) [1] is a distributed learning paradigm that enables multiple clients to collaboratively train a shared model without exchanging their raw data. At round t, the server distributes the current global model θ^{t-1} to N participating clients, denoted by $\{\mathcal{U}_i\}_{i=1}^{N}$, and each client $\mathcal{U}_i$ performs local training on its private dataset $\mathcal{D}_i$. The local model update generated by client $\mathcal{U}_i$ is given by $g_i = \mathcal{OPT}_E\left(\theta^{t-1}, \mathcal{D}_i\right) - \theta^{t-1}$, where $\mathcal{OPT}_E(\theta^{t-1}, \mathcal{D}_i)$ denotes the locally trained model obtained by client $\mathcal{U}_i$ after performing E epochs of training on $\mathcal{D}_i$ starting from θ^{t-1}. The model update g_i is uploaded to the server for aggregation. The server aggregates the received model updates as $\theta^t = \theta^{t-1} + \sum_{i=1}^{N} \frac{|\mathcal{D}_i|}{|\mathcal{D}|} g_i$, where $|\mathcal{D}| = \sum_{i=1}^{N} |\mathcal{D}_i|$ denotes the total number of training samples across all participating clients.

Clustered Federated Learning. Clustered federated learning (CFL) [32] extends conventional FL by grouping clients with similar update characteristics into different clusters. All clients are initialized with the same global model θ^0. After clustering, clients are partitioned into clusters $\{\mathcal{C}^\gamma\}_{\gamma=1}^{\Gamma}$, and the model updates of clients within each cluster are aggregated to obtain the corresponding cluster-level model update. Each cluster then independently performs subsequent model training and aggregation.

B. Top-k Sparsification

Top-k sparsification is a compression technique for reducing communication overhead in FL [21], [22]. Let $g_i \in \mathbb{R}^d$ denote the local model update generated by client $\mathcal{U}_i$. Let $k \in (0, 1]$ denote the sparsification ratio, where kd is assumed to be an integer. The sparsified update is computed as $\tilde{g}_i = \mathrm{TopK}(g_i, kd)$, where $\mathrm{TopK}(\cdot)$ retains the kd entries with the largest magnitudes and sets all remaining entries to zero. The resulting sparse model update $\tilde{g}_i$ is then transmitted to the server.

C. Segmented Cheon-Kim-Kim-Song (CKKS)

Segmented CKKS is a segmented packing framework built upon the approximate CKKS homomorphic encryption scheme [33]. It partitions a high-dimensional vector into multiple segments, each of which is encoded and encrypted independently, thereby alleviating the slot capacity limitation of a single ciphertext while preserving efficient homomorphic computation. As it builds upon CKKS, segmented CKKS relies on the Ring Learning With Errors (RLWE) assumption and supports homomorphic addition, multiplication, and slot rotation over encrypted data.

Let $n = 2^\nu$ denote the ring dimension, and define

$$R = Z\left[X\right] / \left(X^n + 1\right), \quad R_Q = Z_Q\left[X\right] / \left(X^n + 1\right),$$

where $Q = \prod_{l=0}^{L} q_l$ is the modulus chain.Let $m = n/2$ denote the slot capacity of a CKKS ciphertext. For each segment, a complex vector $z = (z_0, z_1, \ldots, z_{v-1}) \in \mathbb{C}^v$ with $v \leq m$ is encoded into a plaintext polynomial $m(X) \in R_Q$ using canonical embedding with scaling factor Δ. Thus, each ciphertext can pack up to m complex values, and homomorphic operations are performed independently on the packed slots. High-dimensional vectors exceeding the slot capacity are represented using multiple ciphertext segments.

Plaintexts are represented as elements of R_Q, while ciphertexts consist of elements over R_Q. Let χ denote the secret key and encryption randomness distribution, and let Ψ denote the error distribution over R_Q. Let $\mathcal{A}$ denote the set of rotation steps required for homomorphic evaluation. Homomorphic addition, ciphertext-ciphertext multiplication, and ciphertext-plaintext multiplication are denoted by $\oplus$, $\otimes$, and $\odot$, respectively.

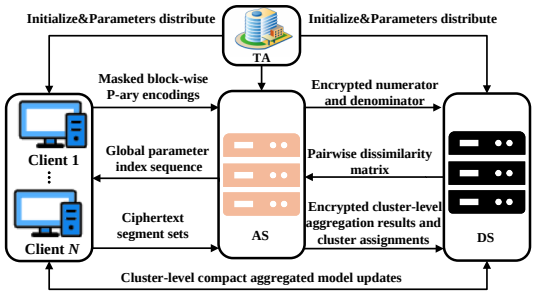


Fig. 1. System model of EPBFL.

The main algorithms of segmented CKKS are summarized below:

$\mathbf{KeyGen}(\lambda) \rightarrow (pk, sk, rlk, \mathcal{RK})$: Given the security parameter λ, sample $a \leftarrow R_Q$ uniformly, $s \leftarrow \chi$, and $e \leftarrow \Psi$. This algorithm generates a public key $pk = (b, a) \in R_Q^2$, where $b = -a \cdot s + e \mod Q$, a secret key $sk = s$, a relinearization key rlk for ciphertext multiplication, and a set of rotation keys $\mathcal{RK} = \{rk_r\}_{r \in \mathcal{A}}$ for slot rotations.

$\mathbf{Encode}(z, \Delta) \rightarrow m(X)$: Given an input vector $z \in \mathbb{C}^v$ and a scaling factor Δ, this algorithm encodes z into a plaintext polynomial $m(X) \in R_Q$ using canonical embedding with scaling factor Δ.

$\mathbf{Enc}(m(X), pk) \rightarrow ct$: Given plaintext polynomial $m(X)$ and public key $pk = (b, a)$, sample $u \leftarrow \chi$, $e_0, e_1 \leftarrow \Psi$, and output ciphertext is computed as:

$$ct = (c_0, c_1) = (u \cdot b + m(X) + e_0, u \cdot a + e_1) \mod Q$$

$\mathbf{Dec}(ct, sk) \rightarrow m'(X)$: Given ciphertext $ct = (c_0, c_1)$ and secret key $sk = s$, this algorithm computes $m'(X) = c_0 + c_1 \cdot s \mod Q$, where the approximate plaintext is recovered.

$\mathbf{Decode}(m'(X), \Delta) \rightarrow z$: Given the plaintext polynomial $m'(X) \in R_Q$ and scaling factor Δ, this algorithm applies inverse canonical embedding and rescales by Δ to output an approximate vector $z \in \mathbb{C}^v$.

$\mathbf{Add}(ct, ct^*) \rightarrow ct_{\text{Add}}$: Given two ciphertexts $ct,\ ct^* \in R_Q^2$, this algorithm performs homomorphic addition $ct_{\text{Add}} = ct \oplus ct^*$.

$\mathbf{Mult}_{rlk}^{ct}(ct, ct^*) \rightarrow ct_{\text{Mult}}$: Given two ciphertexts $ct,\ ct^* \in R_Q^2$ and a relinearization key rlk, this algorithm performs homomorphic multiplication: $ct_{\text{Mult}} = ct \otimes ct^*$.

$\mathbf{Mult}^{pt}(ct, m) \rightarrow ct_{\text{MultPlain}}$: Given a ciphertext $ct \in R_Q^2$ and a plaintext $m \in R_Q$, this algorithm performs homomorphic ciphertext-plaintext multiplication: $ct_{\text{MultPlain}} = ct \odot m$.

$\mathbf{Rot}(ct, r, \mathcal{RK}) \rightarrow ct'$: Given a ciphertext ct encrypting a packed vector z, a rotation step $r \in \mathcal{A}$, and the corresponding rotation key $rk_r \in \mathcal{RK}$, this algorithm outputs a rotated ciphertext ct' satisfying $\text{Dec}(ct', sk) \approx \text{Rot}_r(z)$, where $\text{Rot}_r(z) = (z_r, z_{r+1}, \ldots, z_{v-1}, z_0, \ldots, z_{r-1})$ denotes a cyclic left rotation of the packed vector by r slots.

IV. SCENARIO DESCRIPTION

A. System Model

As shown in Fig. 1, EPBFL consists of four entities:

TA. The TA is responsible for system initialization. It generates the public parameters and cryptographic keys required by the protocol and distributes them to the corresponding entities through secure channels. In addition, the TA establishes a shared random seed for each pair of clients to support the generation of pairwise masks.

Clients. The system consists of N clients, denoted by $\mathcal{U} = \{\mathcal{U}_1, \mathcal{U}_2, \ldots, \mathcal{U}_N\}$. Each client holds a private local dataset and performs local training to generate its local model update. In each communication round, clients apply Top-k sparsification and submit masked block-wise P-ary encodings of their retained parameter indices to the AS. After receiving the global parameter index sequence from the AS, clients construct and encrypt their compact model updates and upload the resulting ciphertext segment sets to the AS. Each client subsequently receives the cluster-level compact aggregated model update from the DS and reconstructs the corresponding d-dimensional cluster-level aggregated model update for the next training round.

Aggregation Server (AS). The AS aggregates and decodes the masked block-wise P-ary encodings to obtain aggregate parameter-index frequencies, selects the global Top-kd parameter indices, and constructs the global parameter index sequence. Based on the ciphertext segment sets received from clients, the AS performs homomorphic dissimilarity computation and sends the encrypted numerator and denominator for each client pair to the DS. It then receives the resulting pairwise dissimilarity matrix from the DS. The AS performs density-based clustering and cluster-wise weighted aggregation over the encrypted compact model updates and sends the encrypted cluster-level aggregation results, together with the corresponding cluster assignments, to the DS.

Decryption Server (DS). The DS decrypts and decodes the encrypted numerator and denominator of the BC-inspired dissimilarity received from the AS, computes the pairwise dissimilarities, and returns the resulting pairwise dissimilarity matrix to the AS. The DS also decrypts and decodes the encrypted cluster-level aggregation results, recovers the cluster-level compact aggregated model updates, and sends each update to the clients in the corresponding cluster.

B. Threat Model

In EPBFL, we consider honest-but-curious clients, Byzantine clients, and honest-but-curious dual servers under the following threat model.

Clients. Honest-but-curious clients faithfully follow the prescribed protocol but may attempt to infer other clients' private information from their protocol views. Byzantine clients may arbitrarily manipulate their local model updates and coordinate with other Byzantine clients to launch attacks, while submitting protocol messages in the prescribed format. Following prior work [11], [34], we consider a benign-majority setting in which at most $N/2$ clients are Byzantine.

Honest-but-Curious and Non-Colluding Dual Servers. Following prior work [11], [12], the AS and the DS faithfully execute the prescribed protocol but may attempt to infer clients' private information. They do not collude with each

other. The AS does not possess the secret key used for CKKS decryption, while the DS does not receive individual clients' ciphertext segment sets or masked block-wise P-ary encodings. We further consider possible collusion between either server and clients and analyze the resulting privacy leakage in the security analysis.

V. Design of EPBFL

A. Overview

EPBFL is an efficient privacy-preserving Byzantine-robust FL framework that enables Byzantine detection and robust aggregation over independently sparsified model updates without revealing individual clients' retained parameter indices or model update values. EPBFL consists of three main components:

Top-k sparsification and privacy-preserving parameter index alignment. In each communication round, clients first perform local training and independently apply Top-k sparsification to retain the parameter indices corresponding to the largest-magnitude model update values. The retained parameter indices are represented using masked block-wise P-ary encodings and uploaded to the AS. By aggregating and decoding these encodings, the AS obtains the aggregate parameter-index frequencies without learning the retained parameter index set of any individual client. Based on the recovered frequencies, the AS selects the global Top-kd parameter indices and constructs a global parameter index sequence. Clients then extract the corresponding parameter values from their local model updates according to this sequence, construct compact model updates, and encrypt them using segmented CKKS encryption.

BC-inspired homomorphic dissimilarity computation. Based on the encrypted compact model updates, the AS homomorphically computes the encrypted numerator and denominator of the BC-inspired dissimilarity for each pair of clients and sends the resulting ciphertexts to the DS. The DS decrypts and decodes the received ciphertexts, computes the pairwise dissimilarities, and sends the resulting pairwise dissimilarity matrix to the AS. This enables pairwise comparison of compact model updates without revealing their plaintext values to the AS.

Density-based Byzantine detection and cluster-wise weighted aggregation. Based on the pairwise dissimilarity matrix, the AS performs density-based clustering to identify density-connected client clusters and isolated outliers without relying on trusted reference model updates. Within each cluster, clients are adaptively weighted according to their average intra-cluster dissimilarities, assigning larger aggregation weights to clients that are more consistent with other clients in the same cluster. The AS then performs cluster-wise weighted aggregation over the encrypted compact model updates and sends the encrypted cluster-level aggregation results to the DS. The DS decrypts the received aggregation results to obtain the cluster-level compact aggregated model updates and sends each update to the clients in the corresponding cluster. Each client then reconstructs the corresponding d-dimensional cluster-level aggregated model update according to the global parameter index sequence for subsequent local training.

B. Initialization

Assume that N clients participate in federated learning, denoted by $\mathcal{U} = \{\mathcal{U}_1, \mathcal{U}_2, \ldots, \mathcal{U}_N\}$. For each pair of clients $\mathcal{U}_i, \mathcal{U}_j \in \mathcal{U}$, the TA generates a shared random seed s_{ij}, where $s_{ij} = s_{ji}$. The seed s_{ij} is securely distributed to $\mathcal{U}_i$ and $\mathcal{U}_j$ and is used to generate block-wise pairwise masks during protocol execution.

The TA determines the public machine-word size w and sets $\beta = \lceil \log_2(N+1) \rceil$, $P = 2^\beta$, and $B = \lfloor w/\beta \rfloor$. The public parameters (P, w, B) are distributed to the AS and all clients.

For homomorphic encryption, the TA generates the public/secret key pair (pk, sk) together with the relinearization key rlk and the set of rotation keys $\mathcal{RK}$. The public key pk is distributed to the AS and all clients, while rlk and $\mathcal{RK}$ are distributed only to the AS for ciphertext multiplication and slot rotation, respectively. The secret key sk is securely delivered to the DS.

C. Masked Block-Wise P-ary Encoding

In communication round t, each client $\mathcal{U}_i$ performs local training on its private dataset $\mathcal{D}_i$ and computes its local model update: $g_i = \mathcal{OPT}_E(\theta^{t-1}, \mathcal{D}_i) - \theta^{t-1}$, where $\mathcal{OPT}_E$ denotes the local optimization operator that performs E local training epochs. Let L denote the number of model layers and d_l the number of parameters in layer l, where $l \in \{1, 2, \ldots, L\}$ and $d = \sum_{l=1}^{L} d_l$. For layers with multi-dimensional parameters, the parameters are flattened into one-dimensional vectors, and their indices refer to the corresponding positions in the flattened vectors.

Each client $\mathcal{U}_i$ applies Top-k sparsification to g_i and retains the kd elements with the largest magnitudes. Let $\mathcal{E}_i^l \subseteq \{1, 2, \ldots, d_l\}$ denote the parameter indices retained by $\mathcal{U}_i$ in layer l.

For each layer l, the parameter indices are divided into $S_l = \lceil d_l/B \rceil$ consecutive blocks. The b-th block, $b \in \{0, 1, \ldots, S_l - 1\}$, is defined as $\mathcal{B}_b^l = \{bB+1, bB+2, \ldots, \min((b+1)B, d_l)\}$. For each block $\mathcal{B}_b^l$, client $\mathcal{U}_i$ constructs the block-wise P-ary encoding $f_{i,b}^l(P) = \sum_{j \in \mathcal{E}_i^l \cap \mathcal{B}_b^l} P^{j-(bB+1)}$.

Client $\mathcal{U}_i$ generates the corresponding pairwise mask

$$\xi_{i,b}^l = \sum_{\mathcal{U}_j \in \mathcal{U},\, i<j} \mathrm{PRG}(s_{ij} \| l \| b \| t) - \sum_{\mathcal{U}_j \in \mathcal{U},\, i>j} \mathrm{PRG}(s_{ij} \| l \| b \| t) \pmod{2^w}.$$

where $\mathrm{PRG}(\cdot)$ denotes a pseudorandom generator whose output is interpreted as an element of $\mathbb{Z}_{2^w}$.

Each client $\mathcal{U}_i$ masks each block-wise P-ary encoding $f_{i,b}^l(P)$ as $\tilde{f}_{i,b}^l(P) = f_{i,b}^l(P) + \xi_{i,b}^l \pmod{2^w}$. Finally, $\mathcal{U}_i$ sends the masked block-wise P-ary encoding set $\tilde{f}_i^l = \{\tilde{f}_{i,b}^l(P)\}_{b=0}^{S_l-1}$ $(l \in \{1, 2, \ldots, L\})$ to the AS.

D. Masked Block-Wise P-ary Decoding

After receiving the masked block-wise P-ary encoding sets, the AS aggregates them to recover the aggregate parameter-index frequencies. For each layer l and block b, the AS computes

$$F_b^l(P) = \sum_{\mathcal{U}_i \in \mathcal{U}} \tilde{f}_{i,b}^l(P) \pmod{2^w}.$$

The AS performs radix-P decomposition on each aggregated block $F_b^l(P)$. For the r-th position in block b, where $r \in \{0,1,\ldots,B-1\}$, the AS computes $a_{b,r}^l = \lfloor F_b^l(P)/P^r \rfloor \bmod P$. The position r in block b corresponds to parameter index $z = bB + r + 1$. For each valid index $z \le d_l$, the AS sets $a_z^l = a_{b,r}^l$, where a_z^l denotes the number of clients that retain parameter index z in layer l.

The AS ranks all parameter indices with $a_z^l > 0$ across all layers according to their aggregate frequencies a_z^l and selects the top kd parameter indices. Ties are broken in ascending order of the layer index and parameter index. For each layer l, the selected parameter indices are arranged in ascending order to form π^l. The resulting global parameter index sequence is denoted by $\pi^g = (\pi^1, \pi^2, \ldots, \pi^L)$, where $\sum_{l=1}^{L} |\pi^l| = kd$. The AS sends π^g to all clients.

E. Compact Model Update Construction and Encryption

After receiving the global parameter index sequence π^g, each client $\mathcal{U}_i$ extracts the corresponding parameter values from its local model update. For each layer l, $\mathcal{U}_i$ extracts the parameter values at the selected indices as

$$\hat{g}_i^l = \left(g_i^l[\pi^l[1]], g_i^l[\pi^l[2]], \ldots, g_i^l[\pi^l[|\pi^l|]]\right).$$

The extracted parameter values from all layers are concatenated into the compact model update

$$\hat{g}_i = Concat(\hat{g}_i^1, \hat{g}_i^2, \ldots, \hat{g}_i^L) \in \mathbb{R}^{d'},$$

where $d' = kd$ denotes the total number of globally selected parameters. Each client $\mathcal{U}_i$ sequentially partitions the compact model update $\hat{g}_i$ into $Y = \lceil d'/m \rceil$ segments, each containing at most m elements, where m denotes the slot capacity of a CKKS ciphertext. Client $\mathcal{U}_i$ encodes and encrypts each segment under pk. For $s = 1, 2, \ldots, Y-1$, $\mathcal{U}_i$ computes the s-th ciphertext segment

$$\mathcal{P}_{i,s} = \mathrm{Enc}_{pk}\left(\mathrm{Encode}\left(\hat{g}_i[(s-1)m+1], \ldots, \hat{g}_i[sm]\right)\right).$$

For the last segment, $\mathcal{U}_i$ computes

$$\mathcal{P}_{i,Y} = \mathrm{Enc}_{pk}\left(\mathrm{Encode}(\hat{g}_i[(Y-1)m+1], \ldots, \hat{g}_i[d'], 0, \ldots, 0)\right),$$

where zero-padding is applied to fill the remaining slots. The resulting ciphertext segment set is $\mathcal{P}_i = \{\mathcal{P}_{i,1}, \mathcal{P}_{i,2}, \ldots, \mathcal{P}_{i,Y}\}$.

Client $\mathcal{U}_i$ uploads $\mathcal{P}_i$ to the AS for subsequent homomorphic dissimilarity computation and cluster-wise weighted aggregation.

F. Homomorphic Dissimilarity Computation

After receiving the ciphertext segment sets from clients, the AS homomorphically computes the encrypted numerator and denominator of the BC-inspired dissimilarity. The DS decrypts the resulting ciphertexts and recovers the pairwise dissimilarity between clients.

Consider two clients $\mathcal{U}_i$ and $\mathcal{U}_j$ with ciphertext segment sets $\mathcal{P}_i$ and $\mathcal{P}_j$ corresponding to the compact model updates $\hat{g}_i$ and $\hat{g}_j$, respectively.

The AS first performs the ciphertext subtraction and ciphertext addition on the corresponding ciphertext segments. For each ciphertext segment $q \in \{1, 2, \ldots, Y\}$, the AS computes $\mathcal{P}_q^{(i,j)-} = \mathcal{P}_{i,q} - \mathcal{P}_{j,q} = \mathrm{Add}\big(\mathcal{P}_{i,q}, \mathrm{Mult}^{pt}(\mathcal{P}_{j,q}, \mathrm{Encode}(-1))\big)$ and $\mathcal{P}_q^{(i,j)+} = \mathcal{P}_{i,q} + \mathcal{P}_{j,q} = \mathrm{Add}\big(\mathcal{P}_{i,q}, \mathcal{P}_{j,q}\big)$.

The AS computes the element-wise squared differences and squared sums through homomorphic multiplication. For each $q \in \{1, 2, \ldots, Y\}$, the AS computes

$$\mathcal{Q}_q^{(i,j)-} = \mathrm{Mult}_{rlk}^{ct}\left(\mathcal{P}_q^{(i,j)-}, \mathcal{P}_q^{(i,j)-}\right),$$
$$\mathcal{Q}_q^{(i,j)+} = \mathrm{Mult}_{rlk}^{ct}\left(\mathcal{P}_q^{(i,j)+}, \mathcal{P}_q^{(i,j)+}\right).$$

The AS performs homomorphic slot rotations and additions to sum all slots within each ciphertext. For each $q \in \{1, 2, \ldots, Y\}$, the AS initializes $\mathcal{S}_{q,0}^{(i,j)\pm} = \mathcal{Q}_q^{(i,j)\pm}$ and, for $r = 0, 1, \ldots, \log_2 m - 1$, iteratively computes

$$\mathcal{S}_{q,r+1}^{(i,j)\pm} = \mathrm{Add}\Big(\mathcal{S}_{q,r}^{(i,j)\pm}, \mathrm{Rot}_{\mathcal{RK}}\Big(\mathcal{S}_{q,r}^{(i,j)\pm}, 2^r\Big)\Big).$$

After the final iteration, let $\mathcal{S}_q^{(i,j)\pm} = \mathcal{S}_{q,\log_2 m}^{(i,j)\pm}$, where every slot of $\mathcal{S}_q^{(i,j)-}$ contains the sum of the squared differences of all parameter values in the corresponding ciphertext segment, while every slot of $\mathcal{S}_q^{(i,j)+}$ contains the sum of the squared sums. The AS aggregates the partial sums from all ciphertext segments: $N_{ij} = \sum_{q=1}^{Y} \mathcal{S}_q^{(i,j)-}$ and $M_{ij} = \sum_{q=1}^{Y} \mathcal{S}_q^{(i,j)+}$, where every slot of N_{ij} contains $\sum_{t=1}^{d'} (\hat{g}_{i,t} - \hat{g}_{j,t})^2$, and every slot of M_{ij} contains $\sum_{t=1}^{d'} (\hat{g}_{i,t} + \hat{g}_{j,t})^2$. Finally, the AS sends N_{ij} and M_{ij} to the DS.

Upon receiving N_{ij} and M_{ij}, the DS decrypts and decodes them as $num_{ij} = \mathrm{Decode}(\mathrm{Dec}_{sk}(N_{ij}))$ and $den_{ij} = \mathrm{Decode}(\mathrm{Dec}_{sk}(M_{ij}))$. Since all slots of num_{ij} and den_{ij} contain the same aggregated values after homomorphic slot summation, the DS uses the first slot to recover the numerator and denominator. The DS computes the final BC-inspired dissimilarity:

$$D_{ij} = \frac{num_{ij}[0]}{den_{ij}[0]} = \frac{\sum_{t=1}^{d'} (\hat{g}_{i,t} - \hat{g}_{j,t})^2}{\sum_{t=1}^{d'} (\hat{g}_{i,t} + \hat{g}_{j,t})^2},$$

where D_{ij} denotes the dissimilarity between clients $\mathcal{U}_i$ and $\mathcal{U}_j$. The DS sends the pairwise dissimilarity matrix $\mathbf{D} = [D_{ij}]_{i,j=1}^{N} \in \mathbb{R}^{N \times N}$ to the AS for subsequent density-based clustering.

G. Robust Clustering and Aggregation

Given the pairwise dissimilarity matrix $\mathbf{D}$, with $D_{ii} = 0$, the AS first determines the neighborhood radius according to the pairwise dissimilarities among clients. For each client $\mathcal{U}_i$ ($i \in \{1, \ldots, N\}$), the AS first computes its median dissimilarity to all other clients as $\rho_i = \mathrm{median}(\{D_{ij} \mid j \neq i\})$. The neighborhood radius is determined by $\varepsilon = \mathrm{median}(\{\rho_i\}_{i=1}^{N})$. Using the precomputed pairwise dissimilarity matrix $\mathbf{D}$ and the radius ε, the AS performs density-based clustering to identify client clusters.

For each client $\mathcal{U}_i$, its ε-neighborhood is defined as $\mathcal{N}_\varepsilon(i) = \{\mathcal{U}_j \mid D_{ij} \le \varepsilon\}$. A client $\mathcal{U}_i$ is identified as a core point if $|\mathcal{N}_\varepsilon(i)| \ge \mathrm{MinPts}$, where $\mathrm{MinPts} > 2$ is a predefined threshold. Density-connected clients are grouped into the same cluster, resulting in Γ clusters, denoted by $\{\mathcal{C}^1, \mathcal{C}^2, \ldots, \mathcal{C}^\Gamma\}$.

Clients that are not assigned to any cluster are regarded as outliers. For each cluster $\mathcal{C}^\gamma$ ($\gamma \in \{1,\dots,\Gamma\}$), the average intra-cluster dissimilarity of client $\mathcal{U}_i \in \mathcal{C}^\gamma$ is computed as $\bar{d}_i^\gamma = \frac{1}{|\mathcal{C}^\gamma|-1}\sum_{\mathcal{U}_j\in\mathcal{C}^\gamma, j\neq i} D_{ij}$.

The aggregation weight of client $\mathcal{U}_i \in \mathcal{C}^\gamma$ is defined as

$$w_i^\gamma = \frac{(\bar{d}_i^\gamma + \delta)^{-1}}{\sum_{\mathcal{U}_j\in\mathcal{C}^\gamma}(\bar{d}_j^\gamma+\delta)^{-1}},$$

where δ is a small positive constant for numerical stability.

For each cluster $\mathcal{C}^\gamma$ ($\gamma \in \{1,\dots,\Gamma\}$), and each client $\mathcal{U}_i \in \mathcal{C}^\gamma$, the AS computes its q-th weighted ciphertext segment, $q \in \{1,\dots,Y\}$, as $\tilde{\mathcal{P}}_{i,q} = \mathrm{Mult}^{pt}\left(\mathcal{P}_{i,q}, \mathrm{Encode}(w_i^\gamma)\right)$.

Next, for each cluster $\mathcal{C}^\gamma$, AS initializes the aggregation of the q-th weighted ciphertext segment as $\mathcal{P}_q^{\gamma,1} = \tilde{\mathcal{P}}_{\mathcal{C}^\gamma[1],q}$, where $\mathcal{C}^\gamma[z]$ denotes the global index of z-th client in cluster $\mathcal{C}^\gamma$. For each $z \in \{2,\dots,|\mathcal{C}^\gamma|\}$, AS iteratively computes $\mathcal{P}_q^{\gamma,z} = \mathrm{Add}\big(\mathcal{P}_q^{\gamma,z-1}, \tilde{\mathcal{P}}_{\mathcal{C}^\gamma[z],q}\big)$. After the final iteration, the q-th aggregated weighted ciphertext segment of cluster $\mathcal{C}^\gamma$ is obtained as $\mathcal{P}_q^\gamma = \mathcal{P}_q^{\gamma,|\mathcal{C}^\gamma|}$. The corresponding aggregated weighted ciphertext segment set is $\mathcal{P}^\gamma = \{\mathcal{P}_1^\gamma, \mathcal{P}_2^\gamma, \dots, \mathcal{P}_Y^\gamma\}$.

The AS sends all cluster-level aggregated weighted ciphertext segment sets $\{\mathcal{P}^\gamma\}_{\gamma=1}^\Gamma$ together with the corresponding cluster assignments to the DS. For each cluster $\mathcal{C}^\gamma$, the DS decrypts and decodes each aggregated weighted ciphertext segment as $z_s^\gamma = \mathrm{Decode}\left(\mathrm{Dec}_{sk}(\mathcal{P}_s^\gamma)\right)$, ($s \in \{1,\dots,Y\}$.) Let $\ell_s = \min\{m, d' - (s-1)m\}$. The DS recovers the valid elements of the compact aggregated model update as

$$\hat{g}^\gamma[(s-1)m + r + 1] = z_s^\gamma[r], \quad r = 0,\dots,\ell_s - 1.$$

The DS concatenates the recovered valid elements from all segments to obtain the cluster-level compact aggregated model update $\hat{g}^\gamma$ and sends it to each client in cluster $\mathcal{C}^\gamma$.

Upon receiving $\hat{g}^\gamma$, each client reconstructs the d-dimensional cluster-level aggregated model update g^γ according to the global parameter index sequence π^g by placing the elements of $\hat{g}^\gamma$ at their corresponding parameter positions and setting the remaining positions to zero. The reconstructed model update g^γ is used to update the cluster model for the next training round.

VI. THEORETICAL ANALYSIS

A. Security Analysis

We analyze the privacy of EPBFL under the threat model described above. We assume that the PRG is computationally secure, CKKS satisfies semantic security, and the AS and the DS do not collude. According to the protocol, we define the prescribed leakage of the AS, the DS, and the clients as follows:

$$\begin{aligned}
\mathcal{L}_{\mathrm{AS}} &= \big(\{F_b^l(P)\}, \pi^g, \mathbf{D}, \{\mathcal{C}^\gamma\}_{\gamma=1}^\Gamma\big),\\
\mathcal{L}_{\mathrm{DS}} &= \big(\{num_{ij}, den_{ij}\}_{i<j}, \{\mathcal{C}^\gamma\}_{\gamma=1}^\Gamma, \{\hat{g}^\gamma\}_{\gamma=1}^\Gamma\big),\\
\mathcal{L}_{\mathcal{U}_i} &= \big(\pi^g, \hat{g}^\gamma\big), \qquad \mathcal{U}_i \in \mathcal{C}^\gamma,
\end{aligned}$$

where $l \in \{1,\dots,L\}$ and $b \in \{0,\dots,\lceil d_l/B\rceil - 1\}$.

Our security analysis focuses on whether an individual client's retained parameter index sets $\{\mathcal{E}_i^l\}_{l=1}^L$ or plaintext compact model update $\hat{g}_i$ are additionally revealed beyond the prescribed leakage above.

Theorem 1. ***(Privacy against the AS).*** *If the AS does not collude with clients or the DS, EPBFL does not reveal an individual client's retained parameter index sets $\{\mathcal{E}_i^l\}_{l=1}^L$ or plaintext compact model update $\hat{g}_i$ to the AS beyond $\mathcal{L}_{\mathrm{AS}}$.*

Proof. Following [34], we use a hybrid argument to simulate the view of the AS.

- **Hybrid 0:** The actual EPBFL protocol.
- **Hybrid 1:** Each $\mathrm{PRG}(s_{ij}\|l\|b\|t)$ is replaced by an independently random value $R_{ij,b}^{l,t} \xleftarrow{\$} \mathbb{Z}_{2^w}$, yielding $\tilde{f}_{i,b}^{l,*}(P)$.
- **Hybrid 2:** Similar to Hybrid 1, but the masked encodings $\{\tilde{f}_{i,b}^{l,*}(P)\}_{i=1}^N$ are replaced by $\{\phi_{i,b}^l\}_{i=1}^N$. The simulator independently samples $\phi_{1,b}^l,\dots,\phi_{N-1,b}^l \xleftarrow{\$} \mathbb{Z}_{2^w}$ and sets $\phi_{N,b}^l = F_b^l(P) - \sum_{i=1}^{N-1}\phi_{i,b}^l \pmod{2^w}$.
- **Hybrid 3:** Each ciphertext segment $\mathcal{P}_{i,q}$ is replaced by $\Phi_{i,q} = \mathrm{Enc}_{pk}(\mathrm{Encode}(0,\dots,0))$.
  The prescribed outputs subsequently received by the AS are generated according to $\mathcal{L}_{\mathrm{AS}}$.

Hybrid-Lemma 1. *Hybrid 0 is computationally indistinguishable from Hybrid 1.*

Proof. Each $\mathrm{PRG}(s_{ij}\|l\|b\|t)$ is replaced by an independent uniform value $R_{ij,b}^{l,t}$, yielding $\zeta_{i,b}^l = \sum_{j>i} R_{ij,b}^{l,t} - \sum_{j<i} R_{ji,b}^{l,t} \pmod{2^w}$. Each pairwise random value appears once with a positive sign and once with a negative sign. Therefore, $\sum_{i=1}^N \zeta_{i,b}^l = 0 \pmod{2^w}$. If Hybrid 0 and Hybrid 1 were computationally distinguishable, the PRG outputs could be distinguished from uniformly random values, contradicting the computational security of the PRG. Hence, $\mathrm{Hybrid}_0 \approx_c \mathrm{Hybrid}_1$. □

Hybrid-Lemma 2. *Hybrid 1 and Hybrid 2 are identically distributed conditioned on $F_b^l(P)$.*

Proof. Fixing all pairwise random values except $\{R_{iN,b}^{l,t}\}_{i=1}^{N-1}$, each $\zeta_{i,b}^l$ for $i < N$ is an affine bijection of an independent uniform variable over $\mathbb{Z}_{2^w}$ and is therefore independently uniform. Consequently, the corresponding masked encodings $\tilde{f}_{i,b}^{l,*}(P)$ for $i < N$ are independently uniform over $\mathbb{Z}_{2^w}$. Since $\sum_{i=1}^N \zeta_{i,b}^l = 0 \pmod{2^w}$, the masked encodings satisfy

$$\sum_{i=1}^N \tilde{f}_{i,b}^{l,*}(P) = F_b^l(P) \pmod{2^w}.$$

Therefore, conditioned on $F_b^l(P)$, once the first $N-1$ masked encodings are fixed, the remaining one is uniquely determined. This is exactly the distribution generated in Hybrid 2. Hence, $\mathrm{Hybrid}_1 \equiv \mathrm{Hybrid}_2$. □

Hybrid-Lemma 3. *Hybrid 2 is computationally indistinguishable from Hybrid 3 for the AS.*

Proof. Since the AS possesses pk but not sk, the semantic security of CKKS makes the actual ciphertext segments computationally indistinguishable from encryptions of zero. The prescribed outputs subsequently obtained by the AS are

included in $\mathcal{L}_{\mathrm{AS}}$ and can be provided by the simulator. Hence, $\mathrm{Hybrid}_2 \approx_c \mathrm{Hybrid}_3$. □

Combining the above hybrids, we have

$$\mathrm{Hybrid}_0 \approx_c \mathrm{Hybrid}_1 \equiv \mathrm{Hybrid}_2 \approx_c \mathrm{Hybrid}_3.$$

Therefore, the view of the AS can be simulated from $\mathcal{L}_{\mathrm{AS}}$. Thus, the AS cannot obtain an individual client's retained parameter index sets $\{\mathcal{E}_i^l\}_{l=1}^L$ or plaintext compact model update $\hat{g}_i$ beyond the prescribed leakage $\mathcal{L}_{\mathrm{AS}}$. □

Theorem 2. ***(Privacy against curious clients).*** *An honest-but-curious client $\mathcal{U}_i$ cannot directly obtain another client's retained parameter index sets or plaintext compact model update beyond the prescribed leakage $\mathcal{L}_{\mathcal{U}_i}$.*

Proof. A client $\mathcal{U}_i \in \mathcal{C}^\gamma$ only receives the global parameter index sequence π^g and the cluster-level compact aggregated model update $\hat{g}^\gamma$. The global parameter index sequence π^g is constructed from the aggregate parameter-index frequencies and does not contain any individual client's retained parameter index sets. Moreover, $\hat{g}^\gamma$ is a weighted aggregation of the compact model updates of the clients in $\mathcal{C}^\gamma$ and does not directly expose an individual client's plaintext compact model update. Since both π^g and $\hat{g}^\gamma$ are included in $\mathcal{L}_{\mathcal{U}_i}$, an honest-but-curious client cannot directly obtain another client's retained parameter index sets or plaintext compact model update beyond the prescribed leakage. □

Theorem 3. ***(Privacy against the DS).*** *If the DS does not collude with the AS or clients, EPBFL does not directly reveal an individual client's plaintext compact model update $\hat{g}_i$ to the DS beyond the prescribed leakage $\mathcal{L}_{\mathrm{DS}}$.*

Proof. For each client pair $(\mathcal{U}_i, \mathcal{U}_j)$, the DS receives N_{ij} and M_{ij} from the AS and decrypts and decodes them as num_{ij} and den_{ij}, respectively. According to the BC-inspired dissimilarity computation, every slot of num_{ij} and den_{ij} contains

$$\sum_{t=1}^{d'}(\hat{g}_{i,t} - \hat{g}_{j,t})^2 \quad \text{and} \quad \sum_{t=1}^{d'}(\hat{g}_{i,t} + \hat{g}_{j,t})^2,$$

respectively. Thus, the DS obtains aggregate statistics over the d' parameters and computes the pairwise dissimilarity D_{ij} using their first slots, rather than directly obtaining the plaintext compact model updates $\hat{g}_i$ and $\hat{g}_j$.

After cluster-wise weighted aggregation, the DS further decrypts the cluster-level aggregated weighted ciphertext segment set $\mathcal{P}^\gamma$ to obtain the cluster-level compact aggregated model update $\hat{g}^\gamma$. Since $\hat{g}^\gamma$ is a weighted aggregation of the compact model updates in $\mathcal{C}^\gamma$, it does not directly expose an individual client's plaintext compact model update.

Therefore, the information available to the DS is limited to the prescribed leakage $\mathcal{L}_{\mathrm{DS}}$, and an individual client's plaintext compact model update $\hat{g}_i$ is not directly revealed beyond this leakage. □

Theorem 4. ***(Privacy against AS–client collusion).*** *If the AS colludes with at most $N-2$ clients, the AS and the colluding clients cannot directly recover the retained parameter index sets $\{\mathcal{E}_i^l\}_{l=1}^L$ or plaintext compact model update $\hat{g}_i$ of a non-colluding client $\mathcal{U}_i$ beyond the prescribed leakage and the information disclosed by the colluding clients.*

Proof. Let $\mathcal{U}_{\mathrm{col}} \subseteq \mathcal{U}$ denote the set of colluding clients, where $|\mathcal{U}_{\mathrm{col}}| \leq N-2$, and let $\mathcal{U}_i \in \mathcal{U} \setminus \mathcal{U}_{\mathrm{col}}$ be a non-colluding client. Since at least two clients remain non-colluding, there exists another non-colluding client $\mathcal{U}_h \in \mathcal{U} \setminus \mathcal{U}_{\mathrm{col}}$ with $h \neq i$.

For each layer l and block b, the pairwise seed s_{ih} is known only to $\mathcal{U}_i$ and $\mathcal{U}_h$. Hence, the AS and the colluding clients cannot remove the pairwise masking component generated from $\mathrm{PRG}(s_{ih}\|l\|b\|t)$ from the mask $\xi_{i,b}^l$ in $\tilde{f}_{i,b}^l(P)$. After subtracting the encodings and masking terms disclosed by the colluding clients, they can obtain only the aggregate encoding of the remaining non-colluding clients, but cannot in general isolate the individual encoding $f_{i,b}^l(P)$ of $\mathcal{U}_i$. Since $f_{i,b}^l(P)$ encodes the retained parameter indices $\mathcal{E}_i^l \cap \mathcal{B}_b^l$ of $\mathcal{U}_i$ in block $\mathcal{B}_b^l$, the complete retained parameter index sets $\{\mathcal{E}_i^l\}_{l=1}^L$ cannot be directly recovered.

The AS also receives the ciphertext segment set $\mathcal{P}_i$ encrypting the compact model update $\hat{g}_i$. Since neither the AS nor the colluding clients possesses the secret key sk, the semantic security of CKKS prevents them from recovering $\hat{g}_i$ from $\mathcal{P}_i$.

Therefore, the retained parameter index sets and plaintext compact model update of an individual non-colluding client are not directly revealed. □

Theorem 5. ***(Privacy against DS–client collusion).*** *If the DS colludes with at most $N-2$ clients, the DS and the colluding clients cannot directly recover the retained parameter index sets $\{\mathcal{E}_i^l\}_{l=1}^L$ or plaintext compact model update $\hat{g}_i$ of a non-colluding client $\mathcal{U}_i$, unless the information available to them uniquely determines $\hat{g}_i$.*

Proof. Let $\mathcal{U}_{\mathrm{col}} = \{\mathcal{U}_{j_1}, \ldots, \mathcal{U}_{j_c}\}$ denote the set of colluding clients, where $c \leq N-2$. Through collusion, the DS can obtain the retained parameter index sets and plaintext compact model updates of the colluding clients. For a non-colluding client $\mathcal{U}_i$, the DS still obtains only the information prescribed by the protocol, including num_{ij}, den_{ij}, the cluster assignments, and the cluster-level compact aggregated model updates. Neither this information nor that disclosed by the colluding clients provides the individual block-wise P-ary encodings $f_{i,b}^l(P)$ of $\mathcal{U}_i$. Hence, the retained parameter index sets $\{\mathcal{E}_i^l\}_{l=1}^L$ of $\mathcal{U}_i$ cannot be recovered.

For each colluding client $\mathcal{U}_{j_r}$, the DS knows $\hat{g}_{j_r}$ and obtains num_{ij_r} and den_{ij_r} from the protocol. According to the BC-inspired dissimilarity computation,

$$den_{ij_r}[0] - num_{ij_r}[0] = 4\sum_{t=1}^{d'} \hat{g}_{i,t}\hat{g}_{j_r,t}.$$

Thus, the DS and the colluding clients can obtain

$$\sum_{t=1}^{d'} \hat{g}_{i,t}\hat{g}_{j_r,t} = \frac{den_{ij_r}[0] - num_{ij_r}[0]}{4}.$$

They can also obtain $num_{ij_r}[0] + den_{ij_r}[0] = 2\sum_{t=1}^{d'} \hat{g}_{i,t}^2 + 2\sum_{t=1}^{d'} \hat{g}_{j_r,t}^2$. Since $\hat{g}_{j_r}$ is known, this further reveals

$\sum_{t=1}^{d'} \hat{g}_{i,t}^2$. However, if these constraints are insufficient to uniquely determine all d' parameter values of $\hat{g}_i$, the plaintext compact model update $\hat{g}_i$ cannot be uniquely recovered. □

B. Correctness Analysis

We analyze the correctness of the masked block-wise P-ary encoding and decoding procedure. In particular, we show that the adopted parameter setting prevents radix-P carry and modular wraparound, thereby enabling the AS to exactly recover the aggregate parameter index frequencies.

Theorem 6. ***(Correctness of block-wise P-ary decoding).*** *For $P = 2^\beta$, where $\beta = \lceil \log_2(N+1) \rceil$, and $B = \lfloor w/\beta \rfloor$, the AS can exactly recover the aggregate parameter-index frequency of each valid parameter index from the aggregated block-wise P-ary encodings.*

Proof. Consider a valid position r of block b in layer l, corresponding to the parameter index $z = bB + r + 1$. According to the block-wise P-ary encoding,

$$f_{i,b}^l(P) = \sum_{j \in \mathcal{E}_i^l \cap \mathcal{B}_b^l} P^{j-(bB+1)}.$$

Therefore, after summing the encodings of all clients, the coefficient of P^r equals the number of clients retaining parameter index z. Since at most N clients participate in the protocol, this coefficient is at most N. By construction, $\beta = \lceil \log_2(N+1) \rceil$ and $P = 2^\beta$, which gives $P > N$. Hence, each aggregate parameter-index frequency can be represented by a single radix-P digit without carry.

Each block contains at most B valid positions, and the coefficient of each position is smaller than P. Thus, the aggregate of the unmasked block-wise P-ary encodings satisfies

$$\sum_{i=1}^{N} f_{i,b}^l(P) \le (P-1) \sum_{r=0}^{B-1} P^r = P^B - 1.$$

Since $B = \lfloor w/\beta \rfloor$ and $P = 2^\beta$, we have $P^B = 2^{\beta B} \le 2^w$, and hence $\sum_{i=1}^{N} f_{i,b}^l(P) < 2^w$. According to the masking procedure,

$$\tilde{f}_{i,b}^l(P) = f_{i,b}^l(P) + \xi_{i,b}^l \pmod{2^w}.$$

Each pairwise masking term appears once with a positive sign and once with a negative sign in the masks of the corresponding two clients. Therefore, $\sum_{i=1}^{N} \xi_{i,b}^l = 0 \pmod{2^w}$. It follows that

$$F_b^l(P) = \sum_{i=1}^{N} \tilde{f}_{i,b}^l(P) \pmod{2^w} = \sum_{i=1}^{N} f_{i,b}^l(P).$$

Since the latter is smaller than 2^w, no modular wraparound affects the aggregated encoding after mask cancellation.

Thus, the AS can recover the aggregate parameter index frequency at position r as

$$a_{b,r}^l = \left\lfloor \frac{F_b^l(P)}{P^r} \right\rfloor \bmod P.$$

For the corresponding parameter index $z = bB + r + 1$, we have $a_z^l = a_{b,r}^l$. Hence, the AS exactly recovers the aggregate parameter index frequency of every valid parameter index. □

VII. Performance Analysis

A. Experimental Setup

1) System Setting: We implement a prototype of EPBFL using PyTorch. For each client, the local test data follow the same data distribution as its local training data, and the reported test accuracy is averaged over all participating clients. All reported results are averaged over 5 independent runs with different random seeds. All experiments are conducted on an Intel i5-14600KF CPU and an NVIDIA RTX 4080 SUPER GPU. We evaluate EPBFL on three benchmark datasets:

MNIST [35]. A handwritten digit dataset containing 70,000 grayscale images of size 28×28 from 10 classes.

Fashion-MNIST [36]. A fashion image dataset containing 70,000 grayscale images of size 28×28 from 10 classes.

CIFAR-10 [37]. An object recognition dataset containing 60,000 color images of size 32×32 from 10 classes.

For MNIST and Fashion-MNIST, we adopt a CNN consisting of two 5×5 convolutional layers, each followed by max pooling, and two fully connected layers. For CIFAR-10, we use ResNet-10. We evaluate all methods under both IID and Non-IID data distributions. For the Non-IID setting, client data are partitioned using a Dirichlet distribution with concentration parameter $\alpha = 0.5$, following prior work [34].

Unless otherwise specified, the number of participating clients is set to $N = 100$. Across all experiments, we set the batch size to 64, the number of local epochs to 5, and the learning rate to 0.01. The Top-k sparsification ratio is set to $k = 0.3\%$, following recent work [17]. For density-based clustering, we set $\mathrm{MinPts} = 3$. We implement CKKS using TenSEAL [38], with the polynomial modulus degree set to $n = 8192$ and the number of slots per ciphertext set to $m = 4096$, following FedPHE [39]. The numbers of communication rounds are set to 25, 50, and 100 for MNIST, Fashion-MNIST, and CIFAR-10, respectively.

2) Attack Settings: We evaluate robustness under several representative Byzantine attacks:

Label-Flipping Attack [40], [41]. Byzantine clients manipulate the labels of their local training data before local training. Specifically, we convert label "1" to "4" for MNIST, "T-shirt/top" to "Shirt" for Fashion-MNIST, and "Automobile" to "Deer" for CIFAR-10.

Random Attack [24]. Byzantine clients replace their genuine local model updates with random updates whose elements are independently sampled from a standard Gaussian distribution.

Sign-Flipping Attack [24]. Byzantine clients reverse the direction of their local model updates by uploading $-g_i$ instead of g_i.

Adaptive Attack [42]. Byzantine clients craft malicious model updates according to the similarity criterion employed by the target defense. For distance-based defenses, malicious updates are crafted such that their distances to benign updates are comparable to the pairwise distances among benign updates. For angular-similarity-based defenses, malicious updates are constructed by scaling benign model updates while preserving their directions.

B. Robustness Evaluation

To evaluate the Byzantine robustness of EPBFL, we compare it with representative robust FL and privacy-preserving Byzantine-robust FL methods. The conventional robust FL baselines include:

- **Krum** [24] selects a client model update based on the pairwise Euclidean distances among client updates.
- **Median** [25] performs coordinate-wise median aggregation over client model updates.
- **AFA** [26] adaptively estimates client reliability and filters suspicious model updates before aggregation.
- **FLTrust** [27] uses a server model update trained on a trusted root dataset as a reference and assigns trust scores to client updates based on cosine similarity.

The privacy-preserving Byzantine-robust FL baselines include:

- **RFed** [11] protects client model updates using CKKS homomorphic encryption and employs scaled dot-product attention to evaluate and downweight malicious updates.
- **ShieldFL** [12] uses two-trapdoor homomorphic encryption for privacy protection and cosine similarity to identify and downweight malicious updates.
- **Qin et al.** [31] combines additive masking with Paillier homomorphic encryption for privacy protection and employs JL-based dimension compression to reduce Byzantine detection overhead, assigning lower weights to updates with greater dissimilarity to a reference update.
- **Split Aggregation** [34] protects client model updates by secret sharing them between two non-colluding servers and applies RSVD-based spectral analysis to identify malicious updates as outliers.
- **Sophon** [43] employs a dual-trust mechanism to suppress malicious updates and adopts a privacy-preserving extension based on two non-colluding servers and secure two-party computation.

1) Impact of Different Byzantine Attacks: Table I reports the final test accuracy of different defense methods under a Byzantine ratio of 0.3 in both IID and Non-IID settings. Compared with the baseline methods, EPBFL generally achieves better performance under Non-IID settings. By clustering clients according to the pairwise dissimilarities of their compact model updates and performing aggregation within each density-connected cluster, EPBFL reduces the interference among clients with heterogeneous update characteristics, which helps maintain higher test accuracy under statistical eterogeneity. Compared with the privacy-preserving Byzantine-robust FL baselines, EPBFL also achieves higher test accuracy in most settings, with particularly clear advantages on CIFAR-10 and Fashion-MNIST.

Moreover, EPBFL maintains high test accuracy under different Byzantine attacks, including the Adaptive attack. By grouping clients according to their pairwise dissimilarities, density-based clustering tends to separate clients with substantially different update characteristics into different clusters or identify isolated clients as outliers, while intra-cluster weighting further reduces the influence of clients with large average dissimilarities. In contrast, Krum and AFA suffer noticeable accuracy degradation under Adaptive attacks, where defense-aware malicious updates are specifically crafted against their robustness criteria. FLTrust also achieves competitive accuracy in several settings with the aid of a trusted root dataset, whereas EPBFL does not require such additional trusted data. Overall, EPBFL achieves the best or competitive accuracy in most settings across the three datasets and four Byzantine attacks.

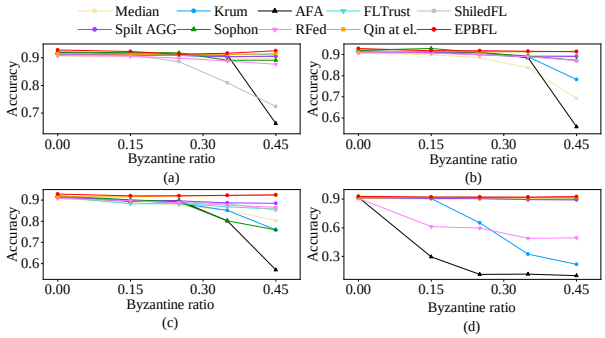


Fig. 2. Test accuracy under different Byzantine ratios on MNIST with Non-IID data under (a)Random acttack, (b)Sign-Flipping acttack, (c)Label-Flipping attack, and (d)Adaptive acttack.

2) Impact of Byzantine Ratio: To evaluate the robustness of different methods with increasing proportions of Byzantine clients, we vary the Byzantine ratio from 0 to 0.45 on MNIST under the Non-IID setting. We consider Random, Sign-Flipping, Adaptive, and Label-Flipping attacks, while keeping all other experimental settings unchanged.

Fig. 2 shows the test accuracy under different Byzantine ratios. Under the Random attack, most methods maintain high accuracy when the Byzantine ratio is low, while ShieldFL and AFA show clear accuracy drops as the ratio increases. EPBFL remains stable and achieves an accuracy above 0.9 even when the Byzantine ratio reaches 0.45. Under the Sign-Flipping attack, the accuracy of Median, Krum, and AFA decreases considerably at high Byzantine ratios, whereas EPBFL remains above 0.9. A similar result is observed under the Label-Flipping attack, where EPBFL maintains an accuracy of approximately 0.92 at a Byzantine ratio of 0.45.

The performance difference is particularly clear under the Adaptive attack. As the Byzantine ratio increases, the accuracy of Krum, AFA, and RFed drops sharply, while EPBFL remains close to 0.92 across the tested Byzantine ratios. These results indicate that EPBFL is less sensitive to an increasing Byzantine ratio and maintains high test accuracy even when 45% of the participating clients are Byzantine.

C. Impact of Sparsification Ratio

Fig. 3 illustrates the convergence behavior of EPBFL under different sparsification ratios. Increasing k results in higher test accuracy and lower loss because more model update parameters are retained. Even with a small sparsification ratio of $k = 0.003$ (0.3%), EPBFL still exhibits stable convergence, with only a moderate decrease in test accuracy compared with larger values of k. Meanwhile, a smaller k reduces the number of parameters included in the compact model update, thereby

TABLE I
FINAL TEST ACCURACY (%) OF DIFFERENT DEFENSE METHODS UNDER A BYZANTINE RATIO OF 0.3.

Dataset	Attack	Robust FL				Privacy-Preserving Byzantine-Robust FL					
		Median	Krum	AFA	FLTrust	ShieldFL	Split Agg	Sophon	RFed	Qin et al.	EPBFL
MNIST IID	No attack	92.86	92.92	92.97	92.96	92.98	92.81	93.77	92.90	**95.75**	95.13
	Label-Flipping	91.37	91.66	91.65	**92.56**	91.70	91.66	91.65	91.28	92.48	91.11
	Random	92.80	92.84	92.90	92.91	86.30	92.93	90.11	88.81	93.01	**93.25**
	Sign-Flipping	91.25	92.91	92.90	92.92	91.74	**92.95**	92.52	90.96	92.77	92.81
	Adaptive	92.80	72.32	9.86	92.32	92.97	91.42	93.26	58.09	92.18	**93.62**
MNIST Non-IID	No attack	90.57	91.18	91.81	91.53	91.29	91.34	92.08	90.80	91.61	**92.86**
	Label-Flipping	87.56	88.47	89.73	88.33	89.23	89.72	89.15	88.85	91.16	**92.00**
	Random	89.81	90.70	90.92	91.34	83.74	90.53	90.72	84.08	91.01	**91.37**
	Sign-Flipping	87.45	90.42	90.65	91.37	89.53	86.54	89.90	84.26	**92.90**	88.95
	Adaptive	89.81	38.54	9.63	91.38	90.53	90.46	90.60	48.46	91.94	**92.42**
CIFAR-10 IID	No attack	50.38	56.88	52.81	67.60	68.54	58.70	59.87	53.05	68.43	**77.70**
	Label-Flipping	36.04	44.96	46.63	66.60	63.81	37.34	52.49	52.68	66.37	**76.47**
	Random	38.22	41.62	47.10	66.55	67.86	45.50	56.51	44.18	68.12	**75.97**
	Sign-Flipping	47.15	43.71	50.54	66.78	57.77	49.74	59.64	35.13	66.68	**75.23**
	Adaptive	38.22	14.49	7.53	66.47	65.82	52.81	58.47	35.44	67.59	**76.21**
CIFAR-10 Non-IID	No attack	46.37	47.15	48.59	49.25	49.54	48.06	48.73	49.23	50.48	**51.51**
	Label-Flipping	35.30	45.16	36.61	47.91	39.37	46.72	45.17	40.36	50.13	**50.72**
	Random	38.22	45.16	47.79	47.99	46.49	46.90	43.09	41.56	50.27	**50.94**
	Sign-Flipping	43.79	46.85	40.32	47.19	41.01	39.41	42.09	38.99	50.13	**50.80**
	Adaptive	36.69	9.83	7.45	48.59	49.35	47.82	46.66	27.44	50.39	**50.73**
Fashion-MNIST IID	No attack	80.79	79.81	83.64	81.87	80.83	79.83	81.24	79.80	85.93	**87.48**
	Label-Flipping	77.43	78.59	78.49	73.49	77.71	78.76	79.19	78.01	85.74	**87.41**
	Random	79.23	53.24	79.67	81.72	75.67	79.83	81.12	76.54	85.91	**87.40**
	Sign-Flipping	76.46	50.96	78.26	81.69	77.27	76.76	78.67	77.45	85.93	**87.43**
	Adaptive	79.42	18.24	10.44	82.04	80.84	78.76	80.72	75.80	85.90	**87.44**
Fashion-MNIST Non-IID	No attack	77.13	77.52	77.09	**83.42**	81.00	78.28	79.23	77.62	84.45	83.22
	Label-Flipping	74.45	76.75	76.21	76.54	76.43	76.71	76.90	75.95	**84.32**	84.12
	Random	77.52	78.12	78.33	82.94	74.45	78.65	81.23	73.69	83.10	**83.25**
	Sign-Flipping	74.29	77.48	76.23	83.01	76.60	72.48	77.01	75.80	82.59	**83.10**
	Adaptive	77.13	36.86	13.48	**83.43**	80.97	76.58	80.37	74.51	82.47	83.23

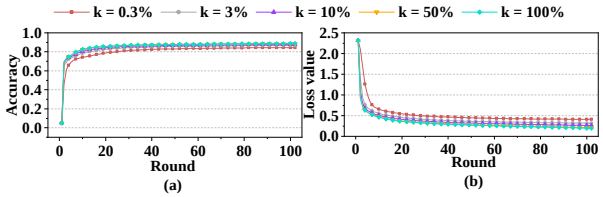


Fig. 3. Impact of different sparsification ratios k on model convergence. (a) Test accuracy. (b) Loss value.

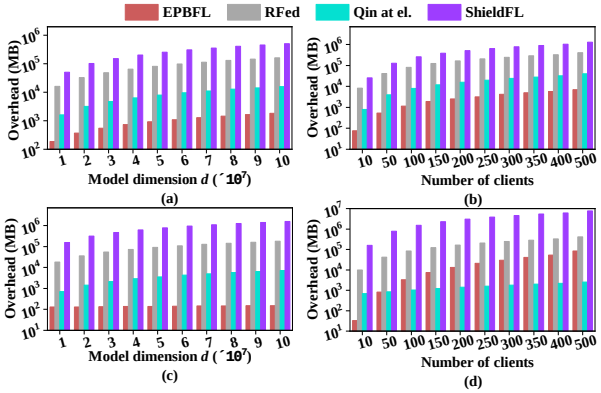


Fig. 4. Communication overhead of different privacy-preserving Byzantine-robust FL schemes. (a) Client-server communication overhead with varying model dimensions for $N = 20$. (b) Client-server communication overhead with varying numbers of clients for $d = 10^7$. (c) Server-to-server communication overhead with varying model dimensions for $N = 20$. (d) Server-to-server communication overhead with varying numbers of clients for $d = 10^7$.

reducing the subsequent encryption, homomorphic computation, and communication overhead. These results indicate that $k = 0.003$ provides a reasonable trade-off between model utility and system efficiency.

D. Efficiency Evaluation

We evaluate the efficiency of EPBFL in terms of communication and computational overhead by comparing it with three privacy-preserving Byzantine-robust FL schemes, including RFed [11], ShieldFL [12], and Qin et al. [31].

1) Communication Overhead: Fig. 4(a) shows that the client-server communication overhead of all schemes increases with the model dimension. ShieldFL incurs the highest communication overhead, followed by RFed and Qin et al., whereas EPBFL achieves the lowest communication overhead. A similar trend is observed in Fig. 4(b) as the number of clients increases. RFed and ShieldFL encrypt and transmit high-dimensional model updates, resulting in considerable ciphertext communication, while Qin et al. reduce the transmitted data through dimension compression. In EPBFL, Top-k sparsification reduces each compact model update to kd parameters, which are further packed into $Y = \lceil kd/m \rceil$ ciphertext segments before transmission. Therefore, EPBFL requires less client-server communication than the compared schemes.

For server-to-server communication, Fig. 4(c) shows that EPBFL incurs the lowest communication overhead as the

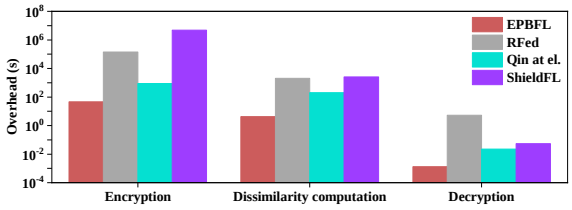


Fig. 5. Computational overhead of different privacy-preserving Byzantine-robust FL schemes in the encryption, dissimilarity computation, and decryption phases.

model dimension increases. In the compared dual-server schemes, intermediate information required for privacy-preserving Byzantine detection is exchanged between the two servers. In EPBFL, the AS first aggregates the segment-wise intermediate results and sends only the encrypted numerator and denominator for each client pair to the DS, avoiding the transmission of segment-wise intermediate ciphertexts. In Fig. 4(d), EPBFL initially incurs lower communication overhead than Qin et al., but exceeds it as N increases, while remaining lower than RFed and ShieldFL. This increase is mainly due to the pairwise dissimilarity computation, which requires communication for $\mathcal{O}(N^2)$ client pairs.

2) Computational Overhead: Fig. 5 compares the computational overhead of different schemes on MNIST using the CNN model in terms of encryption, dissimilarity computation, and decryption. EPBFL incurs the lowest computational overhead among the compared schemes across all three phases.

For encryption, RFed and ShieldFL encrypt high-dimensional model updates, resulting in considerable encryption overhead. Qin et al. reduce the model dimension before privacy-preserving Byzantine detection, but rely on Paillier-based secure computation. In contrast, EPBFL performs encryption over compact model updates and exploits CKKS packing to process multiple model parameters within each ciphertext, resulting in substantially lower encryption overhead.

For dissimilarity computation, RFed and ShieldFL perform cryptographic operations over high-dimensional encrypted model updates, while Qin et al. reduce the computation through JL-based dimension compression. EPBFL performs pairwise dissimilarity computation directly over compact ciphertext segments and exploits CKKS packed operations to process multiple model parameters in parallel. As a result, EPBFL achieves the lowest dissimilarity computation overhead among the compared schemes.

For decryption, EPBFL also incurs the lowest overhead. The AS aggregates the segment-wise intermediate results before transmission, such that the DS decrypts only two ciphertexts for each client pair. After cluster-wise aggregation, the DS additionally decrypts ΓY ciphertext segments to recover the cluster-level compact aggregated model updates.

E. Ablation Study

To evaluate the effectiveness of the key components in EPBFL, we conduct ablation experiments on Fashion-MNIST under the IID setting to reduce the influence of statistical heterogeneity. We consider two variants while keeping all other settings unchanged:

- **Cosine** replaces the proposed BC-inspired dissimilarity with cosine dissimilarity.
- **w/o Clustering** removes the density-based clustering step and treats all clients as a single cluster while retaining the adaptive weighting mechanism.

TABLE II
ABLATION RESULTS ON FASHION-MNIST UNDER THE IID SETTING.

Method	No Attack	Label-Flipping	Random	Sign-Flipping	Adaptive
EPBFL (Full)	**87.48**	**87.41**	**87.40**	**87.43**	**87.44**
Cosine	82.65	79.60	82.74	81.82	9.12
w/o Clustering	82.53	79.03	69.86	77.25	65.09

Table II shows that the full EPBFL maintains an accuracy of approximately 87.4% under all attack settings. Removing density-based clustering reduces the accuracy to 69.86%, 77.25%, and 65.09% under Random, Sign-Flipping, and Adaptive attacks, respectively, demonstrating the importance of clustering in mitigating the influence of malicious updates. Replacing the BC-inspired dissimilarity with cosine dissimilarity also leads to accuracy degradation under all attack settings, with the most significant drop occurring under the Adaptive attack, where the accuracy decreases from 87.44% to 9.12%. These results demonstrate the contributions of both density-based clustering and the BC-inspired dissimilarity to the Byzantine robustness of EPBFL.

VIII. CONCLUSION

In this paper, we presented EPBFL, an efficient privacy-preserving Byzantine-robust FL framework for heterogeneous data. To support Top-k sparsification without exposing individual clients' retained parameter index sets, EPBFL employs masked block-wise P-ary encoding to obtain aggregate parameter-index frequencies and construct a global parameter index sequence for subsequent computation. Segmented CKKS encryption and a BC-inspired dissimilarity are employed to enable efficient pairwise dissimilarity computation over encrypted compact model updates. Based on the resulting dissimilarities, density-based clustering and cluster-wise weighted aggregation mitigate the influence of Byzantine clients without relying on trusted reference data. Experimental results on three benchmark datasets under different data distributions and Byzantine attacks show that EPBFL achieves the best or competitive test accuracy in most settings while substantially reducing communication and computational overhead compared with existing privacy-preserving Byzantine-robust FL schemes.

REFERENCES

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54, 2017, pp. 1273–1282.

[2] H. Gu, L. Fan, X. Tang, and Q. Yang, "Fedcut: A spectral analysis framework for reliable detection of byzantine colluders," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 9, pp. 5905–5920, 2024.

[3] J. Guo, Z. Liu, S. Tian, F. Huang, J. Li, X. Li, K. Konstantin, and J. Ma, "TFL-DT: A trust evaluation scheme for federated learning in digital twin for mobile networks," *IEEE Journal on Selected Areas in Communications*, vol. 41, no. 11, pp. 3548–3560, Nov. 2023.

[4] Y. Xie, M. Fang, and N. Z. Gong, "Model poisoning attacks to federated learning via multi-round consistency," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 15 454–15 463.

[5] H. Pan, Y. Zhang, L. Y. Zhang, S. Adams, A. Kouzani, and S. Khoo, "Performance guaranteed poisoning attacks in federated learning: A sliding mode approach," in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI)*, 2025, pp. 6021–6029.

[6] X. Zhao, J. Shi, Y. Li, J. Huang, and C. Fan, "Good gradients poison your model: Evading defenses in federated learning via boundary-adaptive perturbation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.

[7] C. Dong, J. Weng, M. Li, J.-N. Liu, Z. Liu, Y. Cheng, and S. Yu, "Privacy-preserving and byzantine-robust federated learning," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 2, pp. 889–904, 2024.

[8] P. Tang, X. Zhu, W. Qiu, Z. Huang, Z. Mu, and S. Li, "FLAD: Byzantine-robust federated learning based on gradient feature anomaly detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 4, pp. 3993–4009, 2025.

[9] J. Yu, H. Zhang, Q. Xia, Y. Zou, and Y. Liu, "LPP-FL: A lightweight privacy-preserving federated learning against byzantine attacks on non-IID data," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 11 607–11 622, 2025.

[10] C. Xu, Y. Jia, L. Zhu, C. Zhang, G. Jin, and K. Sharif, "Tdfl: Truth discovery based byzantine robust federated learning," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 12, pp. 4835–4848, 2022.

[11] Y. Miao, X. Yan, X. Li, S. Xu, X. Liu, H. Li, and R. H. Deng, "Rfed: Robustness-enhanced privacy-preserving federated learning against poisoning attack," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 5814–5827, 2024.

[12] Z. Ma, J. Ma, Y. Miao, Y. Li, and R. H. Deng, "Shieldfl: Mitigating model poisoning attacks in privacy-preserving federated learning," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 1639–1654, 2022.

[13] Z. Zhou, J. Zhao, S. Yang, H. Li, T. Ma, and C. Xu, "Community-oriented duplex privacy amplification and active poisoning resistance for heterogeneous federated learning," *IEEE Transactions on Dependable and Secure Computing*, 2025.

[14] H. Guo, H. Wang, T. Song, Y. Hua, R. Ma, X. Jin, Z. Xue, and H. Guan, "SIREN+: Robust federated learning with proactive alarming and differential privacy," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 5, pp. 4843–4860, 2024.

[15] J. Liu *et al.*, "PnA: Robust aggregation against poisoning attacks to federated learning," *ACM Transactions on Intelligent Systems and Technology*, 2024.

[16] C. Zhang, J. Weng, J. Weng, Y. Zhong, J.-N. Liu, and C. Deng, "Robust and secure federated learning with verifiable differential privacy," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 5, pp. 5713–5729, 2025.

[17] S. Xu, Y. Zheng, and Z. Hua, "Harnessing sparsification in federated learning: A secure, efficient, and differentially private realization," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025, pp. 2354–2368.

[18] Z. Chen, S. Yu, F. Chen, F. Wang, X. Liu, and R. H. Deng, "Lightweight privacy-preserving cross-cluster federated learning with heterogeneous data," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 7404–7419, 2024.

[19] M. Egger, R. L. Urbanke, and R. Bitar, "Federated one-shot learning with data privacy and objective-hiding," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 5166–5180, 2025.

[20] G. Zhu, W. Shen, Z. Liu, J. Qin, and J. Ma, "BPFLH: Byzantine-robust privacy-preserving federated learning for heterogeneous data," *IEEE Transactions on Dependable and Secure Computing*, vol. 23, no. 3, pp. 6103–6118, 2026.

[21] Y. Lin, S. Han, H. Mao, Y. Wang, and W. J. Dally, "Deep gradient compression: Reducing the communication bandwidth for distributed training," in *Proceedings of the International Conference on Learning Representations*, 2018.

[22] A. Aji and K. Heafield, "Sparse communication for distributed gradient descent," in *Empirical Methods in Natural Language Processing (EMNLP)*, 2017, pp. 440–445.

[23] F. Kato, Y. Cao, and M. Yoshikawa, "OLIVE: Oblivious federated learning on trusted execution environment against the risk of sparsification," *Proceedings of the VLDB Endowment*, vol. 16, no. 10, pp. 2404–2417, 2023.

[24] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[25] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 5650–5659.

[26] L. Muñoz-González, K. T. Co, and E. C. Lupu, "Byzantine-robust federated machine learning through adaptive model averaging," *arXiv preprint arXiv:1909.05125*, 2019.

[27] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Proceedings of the Network and Distributed System Security Symposium*, 2021.

[28] M. Fang, S. Nabavirazavi, Z. Liu, W. Sun, S. S. Iyengar, and H. Yang, "Do we really need to design new byzantine-robust aggregation rules?" in *Proceedings of the Network and Distributed System Security Symposium*. Internet Society, 2025.

[29] X. Liu, H. Li, G. Xu, Z. Chen, X. Huang, and R. Lu, "Privacy-enhanced federated learning against poisoning adversaries," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 4574–4588, 2021.

[30] B. Zhang, M. Fang, Z. Liu, B. Yi, P. Zhou, Y. Wang, T. Li, and Z. Liu, "Practical framework for privacy-preserving and byzantine-robust federated learning," *IEEE Transactions on Information Forensics and Security*, 2026.

[31] X. Qin, X. Yang, and X. Tang, "Efficient byzantine-robust privacy-preserving federated learning via dimension compression," *IEEE Transactions on Information Forensics and Security*, vol. 21, pp. 2596–2609, 2026.

[32] Y. Xu, Y. Tan, C. Zhang, P. Sun, Y. Zhang, J. Ren, H. Jiang, and Y. Zhang, "Towards privacy-enhanced and robust clustered federated learning," *IEEE Transactions on Mobile Computing*, vol. 24, no. 8, pp. 7171–7188, 2025.

[33] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Advances in Cryptology – ASIACRYPT 2017*, 2017, pp. 409–437.

[34] Z. Lu, S. Lu, Y. Cui, X. Tang, and J. Wu, "Split aggregation: Lightweight privacy-preserving federated learning resistant to byzantine attacks," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 5575–5590, 2024.

[35] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[36] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.

[37] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.

[38] A. Benaissa, B. Retiat, B. Cebere, and A. E. Belfedhal, "Tenseal: A library for encrypted tensor operations using homomorphic encryption," 2021.

[39] Y. Li, N. Yan, J. Chen, X. Wang, J. Hong, K. He, W. Wang, and B. Li, "Fedphe: A secure and efficient federated learning via packed homomorphic encryption," *IEEE Transactions on Dependable and Secure Computing*, 2025.

[40] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," in *Proceedings of the 29th International Conference on Machine Learning*, 2012, pp. 1467–1474.

[41] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, "Data poisoning attacks against federated learning systems," in *Computer Security – ESORICS 2020*, 2020, pp. 480–501.

[42] M. Fang, X. Cao, J. Jia, and N. Z. Gong, "Local model poisoning attacks to byzantine-robust federated learning," in *Proceedings of the 29th USENIX Security Symposium*, 2020, pp. 1605–1624.

[43] X. Gui, G. Yu, J. Wang, Z. Yan, W. Wang, C. Domeniconi, and L. Cui, "Sophon: Byzantine-robust federated learning via dual trust mechanism," *IEEE Transactions on Dependable and Secure Computing*, 2025.